

Colas de Mensajes. Introducción

Las colas de mensajes son un mecanismo de comunicación entre procesos que aportan sistemas operativos UNIX System V, Linux, etc. La idea que subyace bajo las colas de mensajes es parecida a la de las tuberías, pero permiten una mayor versatilidad.

Las colas de mensajes son gestionadas por el núcleo del SO, es decir, es el SO el que proporciona los mecanismos necesarios para que las escrituras y las lecturas en la misma sean síncronas.

En general, en una cola pueden escribir y leer varios procesos. Las operaciones de lecturas son gestionadas siguiendo una política *FIFO* (*First Input First Output*), es decir, el primer mensaje que entra es el primer mensaje que sale. Para hacer que la implementación sea más flexible, se ha permitido hacer lecturas filtradas por el tipo de mensaje, con lo cual, se rompe la filosofía *FIFO*, aunque ésta se sigue manteniendo para todos los mensajes de un mismo tipo. Si se hace una operación de lectura sin especificar el tipo de mensaje, se está forzando a leer el primer mensaje que haya en la cola.

Las llamadas para trabajar con colas de mensajes son: **msgget**, para crear una cola o acceder a una ya existente; **msgctl**, para acceder y modificar la información de control que el núcleo asocia a cada cola de mensajes; **msgsnd**, para escribir/enviar un mensaje en la cola, y **msgrcv**, para leer/recibir un mensaje de la cola.

1. Llamadas del sistema relacionadas con colas de mensajes

1.1. Creación/Acceso de una cola de mensajes: **msgget**

Para crear una cola de mensajes deberemos utilizar la llamada al sistema **msgget**:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgget (key_t key, int msgflg);
```

Si todo va bien devuelve el identificador de la cola que ha creado, en caso contrario, devuelve -1 y en **errno** el código del error.

key es una llave donde todos los procesos que quieran acceder a la cola deben tener la misma

msgflg es una máscara de bits cuyos valores son los mismos que se han visto en el boletín de semáforos.

IPC_CREAT Si este flag está activo, se creará la cola de mensajes, en caso de que no haya sido ya creada.

IPC_EXCL Esta bandera se utiliza junto con **IPC_CREAT**, para lograr que **msgget** de un error en el caso de que se intente crear una cola de mensajes que ya exista. En este caso, **msgget** devolvería -1 e inicializaría la variable **errno** con un valor **EEXIST**.

Los **permisos son** los 9 bits menos significativos de **msgflg**. Y corresponden a indican los permisos de la cola de mensajes. Los tres primeros representan los permisos para el propietario del archivo. Los tres siguientes son los permisos para el grupo del archivo y los tres últimos son los permisos para el resto del mundo o otros (por ejemplo **rw-rw-rw**: r: read w write x execution). Las nueve posiciones de permisos son en realidad un bit que o está encendido (mostrado con su letra correspondiente) o está apagado (mostrado con un guión -), así que, por ejemplo, permisos como **rw-rw-r--**, indicaría que los permisos del propietario (**rw**) puede leer, escribir y ejecutar el archivo, el grupo (o sea los usuarios que estén en mismo grupo del archivo) (**rw**-) podrá leer y escribir pero no ejecutar el archivo, y cualquier otro usuario del sistema (**r--**), solo podrá leer el archivo, ya que los otros dos bits de lectura y ejecución no se encuentran encendidos o activados.

Ejemplo 1 Las siguientes líneas de código crean una cola de mensajes.

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define PERMISOS S_IRUSR | S_IWUSR

int msqid;
key_t llave;
llave= 0x1234;    //puede ser cualquier número o generarla con: llave= ftok("prueba", 'A');

msqid = msgget (llave, IPC_CREAT | PERMISOS);
if (msqid == -1)
    /* Error al crear la cola de mensajes */
```

1.2. Operaciones con colas de mensajes: **msgsnd** y **msgrcv**

Para enviar un mensaje utilizando una cola de mensajes utilizamos la llamada al sistema **msgsnd** y para leer un mensaje de la cola se utiliza **msgrcv**:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgsnd (int msqid, void *msgp, int msgsz, int msgflg);
int msgrcv (int msqid, void *msgp, int msgsz, long msgtyp, int msgflg);
```

donde:

msqid: es el descriptor de la cola de mensajes a utilizar.

msgp: apunta a la zona de memoria que contiene los datos que vamos a enviar o recibir. Cómo esté organizada esta zona de memoria es cuestión que se ha dejado al programador. El único requisito que ha de cumplir es que el primer campo debe ser de tipo long y hay que utilizarlo para identificar el tipo de mensaje. No hay ningún tipo definido por el sistema, por lo tanto la tipificación de mensajes depende del programador.

msgsz: es el tamaño en bytes que ocupa el mensaje a enviar o recibir. En este tamaño no se incluyen los bytes del long que se utiliza para indicar el tipo del mensaje.

msgtyp: solamente se utiliza en **msgrcv** e indica el tipo de mensaje que se quiere leer. De forma que si:

- **msgtyp** = 0, entonces se lee el primer mensaje que haya en la cola.
- **msgtyp** > 0, entonces se lee el primer mensaje de tipo **msgtyp** que haya en la cola.
- **msgtyp** < 0, entonces se lee el primer mensaje cuyo tipo sea menor o igual al valor absoluto de **msgtyp** y sea el de menor valor de los que cumplan esta condición.

msgflg: es un conjunto de bits que tienen distinto significado dependiendo de si aparece en una operación de lectura o de escritura. En el caso de una escritura (**msgsnd**) se tiene:

- Si la cola está llena y el bit IPC_NOWAIT está activo, **msgsnd** devuelve el control inmediatamente, retornando un -1. Es decir, el mensaje no se envía.
- Si la cola está llena y el bit IPC_NOWAIT no está levantado, el proceso suspende su ejecución hasta que haya espacio en la cola y pueda enviar el mensaje.

Para la llamada `msgrcv`, se tiene lo siguiente:

- Si no hay ningún mensaje del tipo especificado en la cola y el bit IPC NOWAIT está activado, la llamada devuelve el control inmediatamente retornando un -1. Es decir, el mensaje no se recibe.
- Si no hay ningún mensaje del tipo especificado en la cola, y el bit IPC NOWAIT no está levantado, el proceso suspende su ejecución esperando hasta que haya un mensaje del tipo deseado que poder recibir.

Si `msgsnd` se ejecuta sin ningún tipo de problema, devuelve 0. Si `msgrcv` se ejecuta sin errores, retorna el número total de bytes que ha recibido en `msgp`. Como ya hemos visto antes, este tamaño no incluye los bytes que ocupa el campo del tipo del mensaje.

En el caso de que haya algún tipo de problema, las llamadas devolverán un -1, y en `errno` el código del error.

Ejemplo 2 *La siguiente porción de código muestra cómo enviar y recibir un mensaje utilizando las dos llamadas anteriores.*

```
int msqid;
struct mensaje{
    long tipo;
    char cadena[LON_CADENA];
};
...
int longitud = sizeof(mensaje) - sizeof (mensaje.tipo);
...
/* Enviar mensaje de tipo 1 */
mensaje.tipo = 1;
strcpy(mensaje.cadena, "PRUEBA");
if ( msgsnd(msqid, &mensaje, longitud, 0) == -1)
    /* Error al enviar el mensaje */
...
/* Recibir mensaje de tipo 1 */
if (msgrcv (msqid, &mensaje, longitud, 1, 0 ) == -1)
    /* Error al recibir el mensaje */
```

1.3. Control de las colas de mensajes: `msgctl`

Para leer y modificar la información estadística y de control de una cola se utiliza la llamada `msgctl`. Su declaración es la siguiente:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgctl (int msqid, int cmd, struct msqid_ds *buf);
```

donde:

`msqid`: es el identificador de la cola de mensajes sobre la que se va a actuar.

`cmd`: es la operación a realizar. Sus posibles valores están definidos con las siguientes constantes:

IPC_STAT Lee el estado de la estructura de control de la cola y lo devuelve en la zona de memoria apuntada por `buf`.

Esta estructura está definida en <msg.h> como sigue:

```
struct msqid_ds {
    struct ipc_perm    msg_perm;    /* operation permission struct */
    struct msg         *msg_first;  /* ptr to first message on q */
    struct msg         *msg_last;   /* ptr to last message on q */
    msglen_t           msg_cbytes;  /* current # bytes on q */
    msgqnum_t          msg_qnum;    /* # of messages on q */
    msglen_t           msg_qbytes;  /* max # of bytes on q */
    pid_t              msg_lspid;   /* pid of last msgsnd */
    pid_t              msg_lrpid;   /* pid of last msgrcv */
    time_t             msg_stime;   /* last msgsnd time */
    time_t             msg_rtime;   /* last msgrcv time */
    time_t             msg_ctime;   /* last change time */
};
```

IPC_SET Inicializa los campos de la estructura de control de la cola. El valor de estos campos lo toma de la estructura apuntada por buf.

IPC_RMID Borra del sistema la cola de mensajes cuyo identificador es msqid.

Ejemplo 3 La siguiente línea de código borra una cola de mensajes que previamente fue creada con msgget.

```
msgctl( msqid, IPC_RMID, 0);
```

Ejemplo 4 La siguiente función comprueba si existen mensajes pendientes de ser leídos en una cola:

```
bool Mensaje_Pendiente(int mb)
{
    msqid_ds buf;
    msgctl(mb, IPC_STAT, &buf);
    return ( buf.msg_qnum != 0 );
}
```